



Embodied AI Practicals II: ROS, Project 3 and Presentation

Course AIAA 4220

Changhao Chen
Assistant Professor
HKUST (GZ)

Roadmap

- 1. ROS**
 - 2. Project 3 Release**
 - 3. Presentation**
 - 4. Q&A on Project 2**
- 

The Robot Operating System (ROS)

Writing Software for Robotics

- Robotics requires very complex software
- We deal with the complexity through modularity
- Different from in Simulators, where you can simply control a robot by some API
 - In real-world, usually each part of the robots (camera, arms, other sensors) are connected via some network

<https://wiki.ros.org/ROS/Tutorials>

Robot Operating System (ROS)

- An implementation of pub/sub geared towards robotic applications and that is **network-aware**
- Lots of open-source software shared by the community:
 - SLAM (gmapping, amcl)
 - Vision (OpenCV*, PCL, OpenNI)
 - Arm Navigation (MoveIt, MoveIt2)
 - Simulation (Gazebo, CARLA/Isaac Sim also support it)
- Most robots design their APIs in the ROS framework
 - Unitree robots, etc.

* Example: <https://automaticaddison.com/working-with-ros-and-opencv-in-ros-noetic/>

ROS History

- ~2006, Former Google VPs decide to work on Robots
 - Create a company called Willow Garage
 - From this org we get OpenCV, PCL, ROS, PR2 Robot, and many spin out
- 2007-Today
 - Willow awards 11 \$400k PR2 robots to Universities
 - OSRF (Open Source Robotics Foundation) created to maintain ROS and Gazebo
 - 2017 ROS 2 Begins to move ROS out of the lab
 - Address security and robustness concerns
 - ROS is everywhere!



The Main Software Components

- The core of ROS is the ROS publisher/subscriber bus
 - A **topic** has a message type that is published on the bus. These messages are defined in a yaml file and define the serialization/deserialization format for ROS messages
 - Message types: abstraction away from specific hardware
 - Camera image
 - Laser scan data
 - Motion control
 - ROS has a lot of built in message types. There are lots of pre-defined messages for controlling a robot, distributing sensor data, and understanding the geometry of your robot
 - ROS publishers produce messages slowly or as quickly as they need to
 - A ROS subscriber, subscribes to a topic and then does things with the information

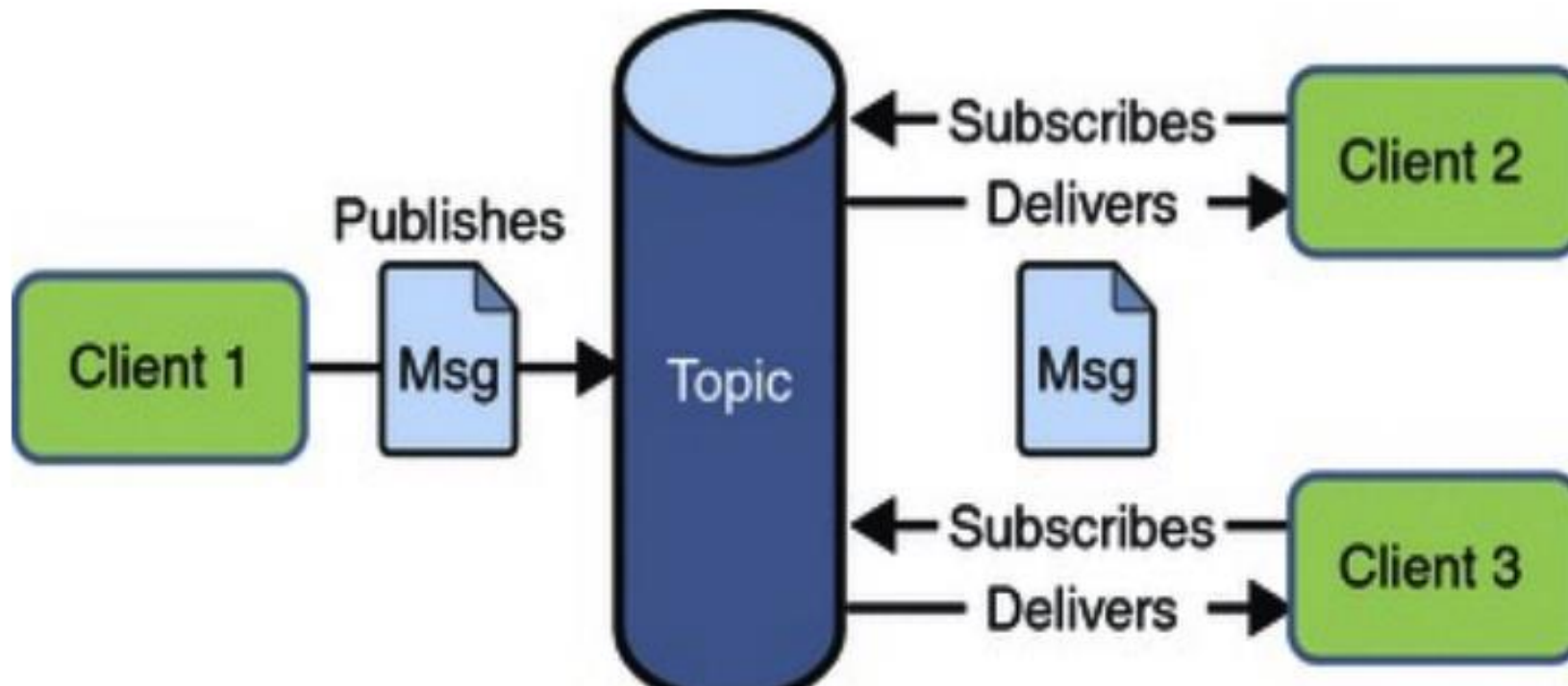
The Publisher/Subscriber Design Pattern

- We divide our software into individual components
- We define “**topics**” (think chat rooms) where components can broadcast information to anyone listening
 - Or a sensor like LiDAR or RGB camera in AD system
- Each component can:
 - Publish: send messages to a topic regardless of whether someone is listening or not
 - Subscribe: receive messages on a topic if anyone is sending them regardless of who

The Publisher/Subscriber Design Pattern

- Example graph for topic -> publish/subscribe

So this means that suppose we have a RGB camera sending out RGB stream to a topic, clients like the robot leg controller and a person detection perception model can access this at the same time

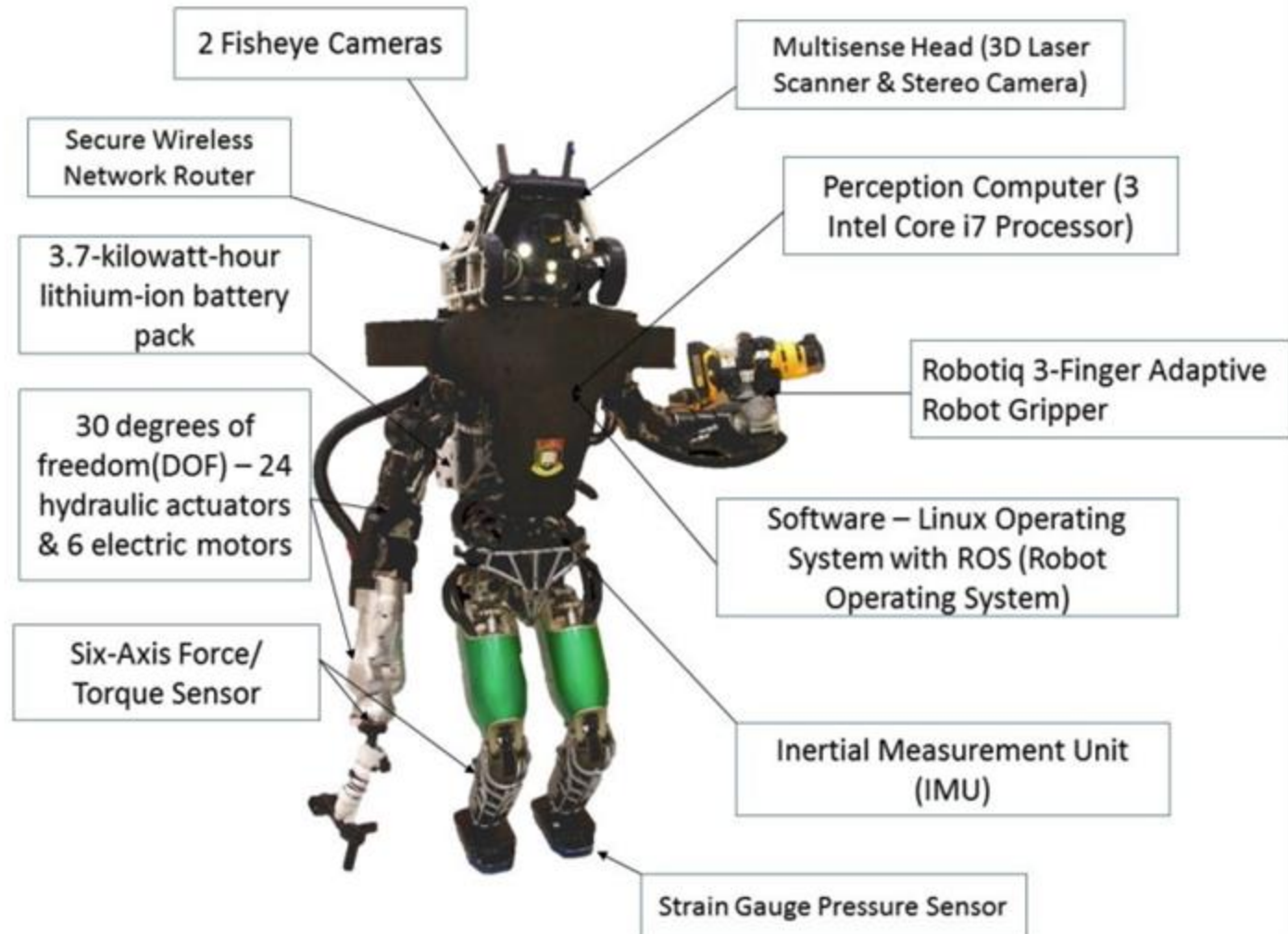


ROS Node

- A process (typically Python or C++) that runs some computation
- The “fundamental” building block
- A ROS **node** can **publish** to one or more **topics**
- That same node can subscribe to other topics
- Many nodes subscribe to topics, process the data, and publish the results
- Run them using `roslaunch <package> <node>`
- Initialize using `rospy.init_node()`
- Some notes
 - Processes / Threads ==> ROS Nodes -- A ROS Node is a self contained execution process, like a program. ROS is really a lot of tooling for running a bunch of programs in parallel

ROS Node

- Sensors and actuators are wrapped in self-contained, reusable software containers called “nodes”

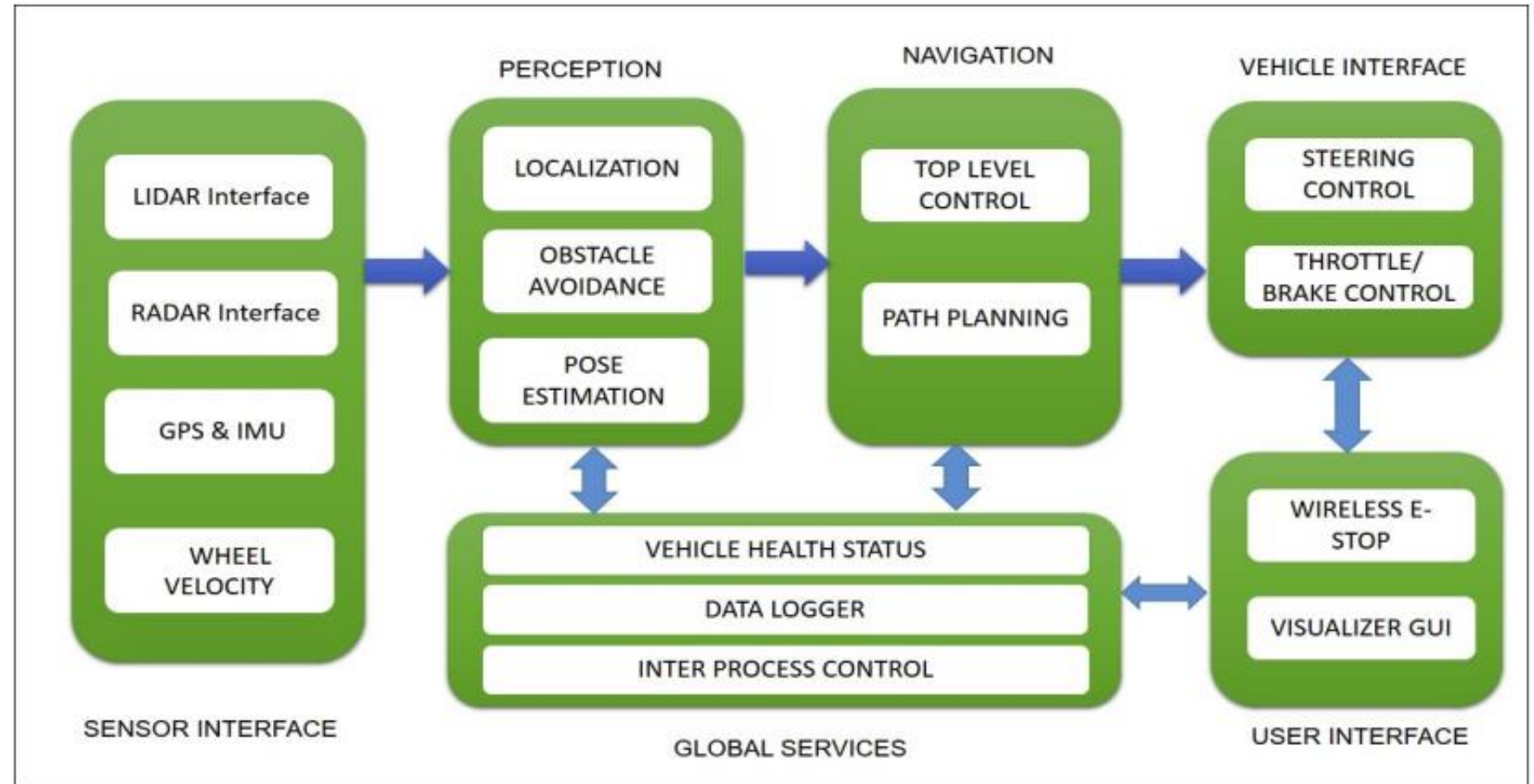


* Robot From DARPA Challenge

ROS

ROS Node

- Higher level operations also become nodes in the ROS computational architecture
 - Perception
 - Navigation



More Concrete Node Examples

- LiDAR node publishes laser scan arrays
- Camera node publishes RGB images (+depth if RGBD) and camera info (resolution, distortion coefficients)
- Mobile robot controller publishes odometry values (e.g. x-y coordinates and velocities, +z for UAVs or underwater vehicles)
- Navigation node **subscribes** to LiDAR and odometry messages, **publishes motion control** messages

ROS Master

- A process that is in charge of coordinating nodes, publishers and subscribers
- Also provides a global parameter server
- Exactly one of them running at any time
- Messages do NOT go through Master (i.e. peer-to-peer)
- Nodes will not be able to find each other without Master
- **All of the above not true in ROS 2 (no more master!)**
 - **Some legacy robots still use ROS 1..**

Sending Messages

```
pub = rospy.Publisher()  
msg = ...  
pub.publish(msg)
```

ROS Messages

- You can check if you are sending messages using the rostopic command line tool:

`rostopic list` – lists all the active topics

`rostopic echo <topic>` – prints messages received on <topic>

`rostopic hz <topic>` – measures topic publishing rate

In ROS 2, the cmd is
\$ `ros2 topic list`

Receiving Messages

```
rospy.Subscriber("chatter", String, callback)  
def callback(msg): ...
```

“chatter” is the topic name

ROS Node - Subscriber

- python code example (rospy)

The callback function is called every time a message is received

```
#!/usr/bin/env python
import rospy
from std_msgs.msg import String

def callback(msg):
    rospy.loginfo("Received: %s", msg.data)

def listener():
    rospy.init_node('listener', anonymous=True)

    rospy.Subscriber("chatter", String, callback)

    rospy.loginfo("Listening on the chatter topic...")

    rospy.spin()

if __name__ == '__main__':
    listener()
```

Using the low-level Communication Tool - DDS

- ROS2 actually uses DDS - So Unitree robots does not use ROS2 but DDS instead
- I like it since it has much smaller overhead compared to ROS2
 - We actually only need Publisher and Subscriber over network - DDS provides them!
 - Here I show you an example code to control the Unitree Dex3 robot hands
 - You need the Subscriber to get the joint states of the hand - is it open? Is it grabbing something?
 - You need the Publisher to send the joint targets to the hand - thumb and index finger touch now!
 - That's it! The rest will be to figure out what joint targets to send at 100Hz, etc.

You don't need
ROS2 to control
Unitree robots.
Use DDS to avoid
overhead

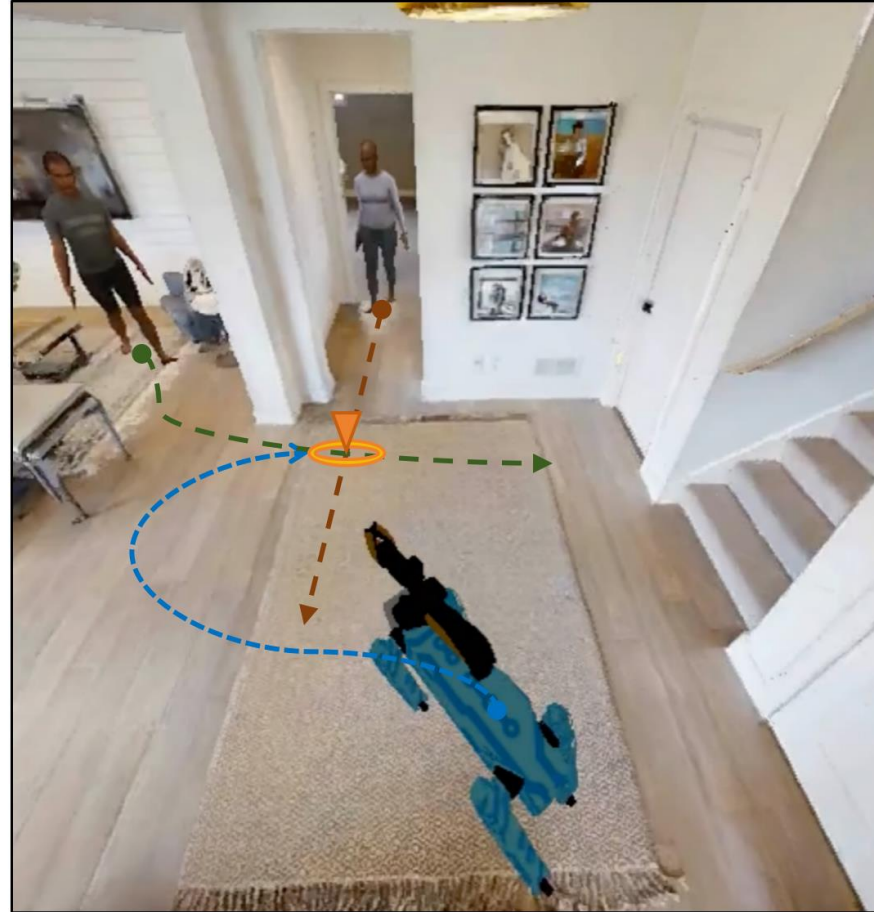
```
# initialize handcmd publisher and handstate subscriber
self.LeftHandState_subscriber = ChannelSubscriber(kTopicDex3LeftState, HandState_)
self.LeftHandState_subscriber.Init()
self.RightHandState_subscriber = ChannelSubscriber(kTopicDex3RightState, HandState_)
self.RightHandState_subscriber.Init()

self.LeftHandCmb_publisher = ChannelPublisher(kTopicDex3LeftCommand, HandCmd_)
self.LeftHandCmb_publisher.Init()
self.RightHandCmb_publisher = ChannelPublisher(kTopicDex3RightCommand, HandCmd_)
self.RightHandCmb_publisher.Init()
```

Social Navigation - background

Navigate safely and efficiently in **crowded** space.

- Autonomous agents
- Dynamic, human-populated indoor environments
- RGBD-based preception



(a) Third-person RGB Perspective



(b) Depth Image



(c) Top-down Map

Project Website

Project website:

https://github.com/Zeying-Gong/aiaa4220_hw3.git

Detailed training, evaluation, visualization guidelines 🧑 🏠

EvalAI:

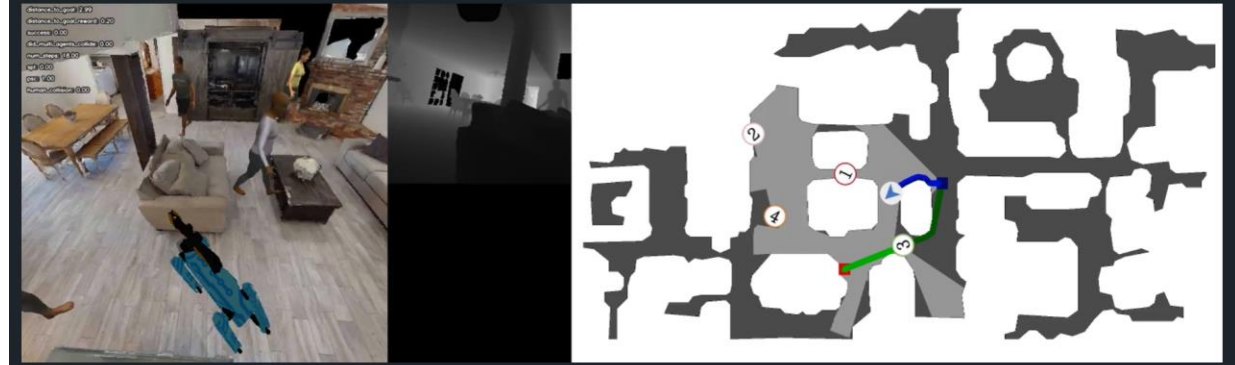
<https://eval.ai/web/challenges/challenge-page/2650/overview>

Submit your solution and see your rank 🏆

AIAA 4220 - Homework 3: Social Navigation with 3D Simulator

Web Falcon ICRA 2025 Submit EvalAI License Apache 2.0

This is the homework 3 for AIAA 4220. We will perform a **social navigation task** on the Habitat simulator.



Social Navigation Challenge

Organized by: socialnav_hw

Starts on: Sep 21, 2025 12:00:00 AM CST (GMT + 8:00)

Ends on: Dec 16, 2025 7:59:59 PM CST (GMT + 8:00)

Social Navigation

Human-Robot Interaction

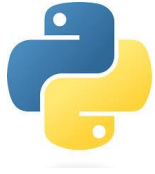
Habitat 3D Simulator

Prerequisites



Linux

- **Linux**
 - Refer to Project 2



- **Python & Pytorch**
 - Refer to Project 2



git

- **Git**
 - Code manage



docker

- **(Optional) Docker**
 - An open-source project to easily create lightweight, portable, self-sufficient containers from any application



HABITAT

- **3D Simulator: Habitat**
 - A high-performance physics-enabled 3D simulator with support for

Docker Installation

Install Docker Desktop on Mac

<https://docs.docker.com/desktop/setup/install/mac-install/>

Install Docker Desktop on Windows

<https://docs.docker.com/desktop/setup/install/windows-install/>

Install Docker Desktop on Mac

Page options ▾

Docker Desktop terms

Commercial use of Docker Desktop in larger enterprises (more than 250 employees or more than \$10 million USD in annual revenue) requires a [paid subscription](#).

This page provides download links, system requirements, and step-by-step installation instructions for Docker Desktop on Mac.

Docker Desktop for Mac with Apple silicon

Docker Desktop for Mac with Intel chip

Install Docker Desktop on Windows

Page options ▾

Docker Desktop terms

Commercial use of Docker Desktop in larger enterprises (more than 250 employees OR more than \$10 million USD in annual revenue) requires a [paid subscription](#).

This page provides download links, system requirements, and step-by-step installation instructions for Docker Desktop on Windows.

Docker Desktop for Windows - x86_64

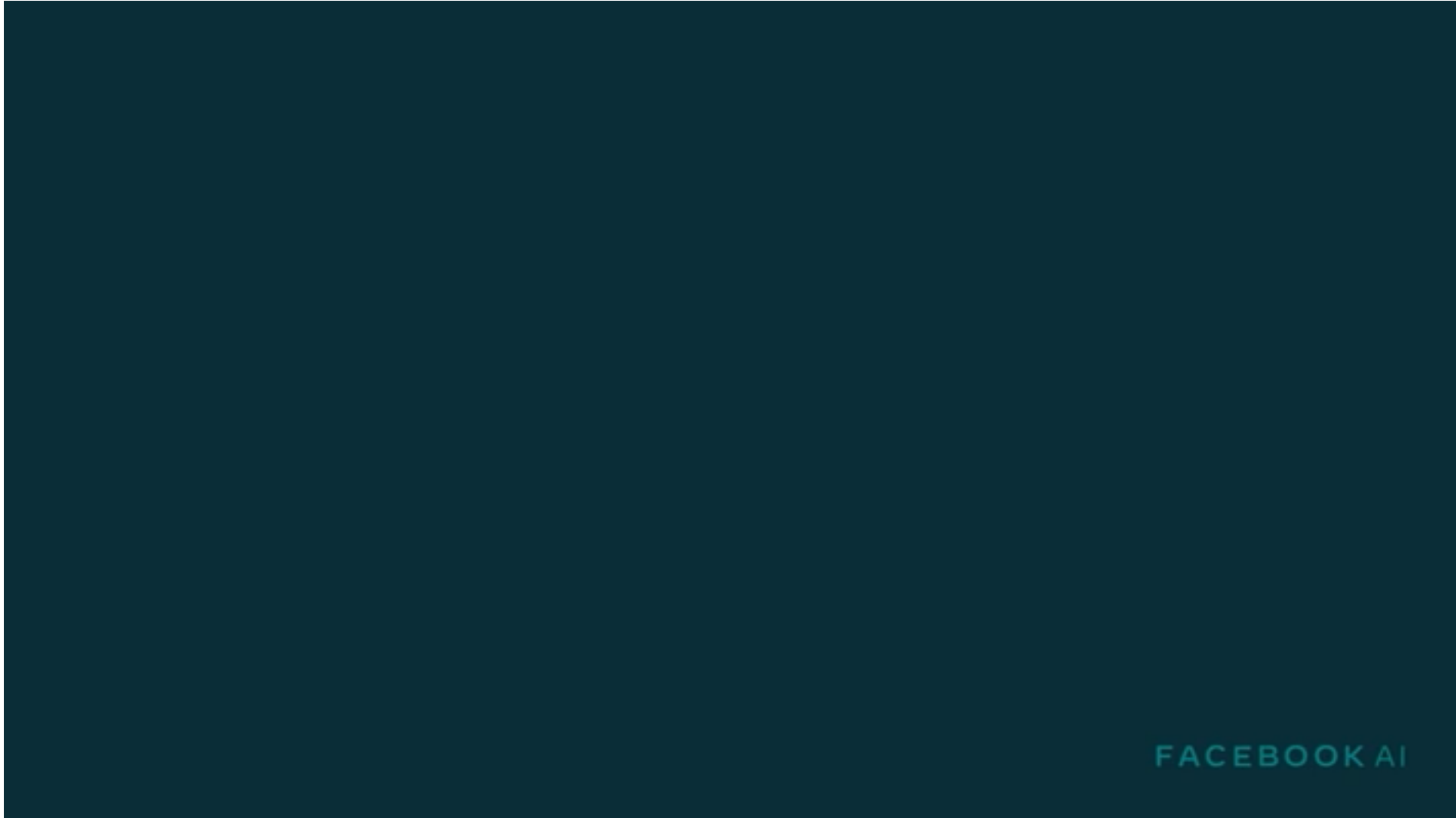
Docker Desktop for Windows - x86_64 on the Microsoft Store

Docker Desktop for Windows - Arm (Early Access)

Docker Basic Usage

Pull Image	<code>docker pull busybox</code>
Run Container	<code>docker run busybox</code>
Shows all containers that are running	<code>docker ps -a</code>
Run container in interactive terminal	<code>docker run busybox -it sh # /bin/bash</code>
Remove container	<code>docker rm {CONTAINER_ID}</code>
Shows images	<code>docker image ls</code>

Habitat: physics-enabled 3D simulator



Video download from <https://github.com/facebookresearch/habitat-sim>

Environment Setup

1. Open a terminal

2. Clone the code

```
git clone https://github.com/Zeying-Gong/aiaa4220_hw3.git
```

If you are using Docker

3. Pull the image

```
docker login  
docker pull zeyinggong/robosense_socialnav:v0.7
```

4. Run the container

```
sudo docker run --name homework -it --gpus all \  
--network host --runtime=nvidia --entrypoint /bin/bash \  
-w /app/Falcon -v {YOUR_LOCAL_PATH}/aiaa4220_hw3/Falcon/:/app/Falcon \  
zeyinggong/robosense_socialnav:v0.7
```

5. Activate python virtual environment

```
source activate falcon
```

Environment Setup

If not using Docker

3. Install python virtual environment by yourself

```
conda create -n habitat python=3.9 cmake=3.14.0
conda activate habitat
# if you have a screen
conda install habitat-sim withbullet -c conda-forge -c aihabitat
# if not
conda install habitat-sim withbullet headless -c conda-forge -c aihabitat
pip install habitat-lab
```

4. Install custom packages

```
cd aiaa4220_hw3/Falcon/ habitat-lab
pip install -e .

cd ../habitat-baselines
pip install -e .
```

Download the Data

TL,DR Download everything from our campus server (need campus network)

```
wget -c http://office.precognition.team/shares/aiaa4220_hw3_data_v2.tgz
```

Download one-by-one

1. Download Scene Datasets

```
python -m habitat_sim.utils.datasets_download --username <api-token-id> --  
password <api-token-secret> --uids hm3d
```

2. Download Task Datasets from huggingface

```
git lfs install  
mkdir -p data/datasets/pointnav  
cd data/datasets/pointnav  
git clone https://huggingface.co/datasets/zgong313/aiaa4220-social-nav-dataset  
cd aiaa4220-social-nav-dataset  
mv * ..  
cd ..  
rm -rf aiaa4220-social-nav-dataset
```

Download the Data

3. Download Multi-agent Dependencies

```
cd ../../../../  
python -m habitat_sim.utils.datasets_download --uids habitat_humanoids  
hab3_bench_assets hab_spot_arm
```

4. Download Leg Animation Data

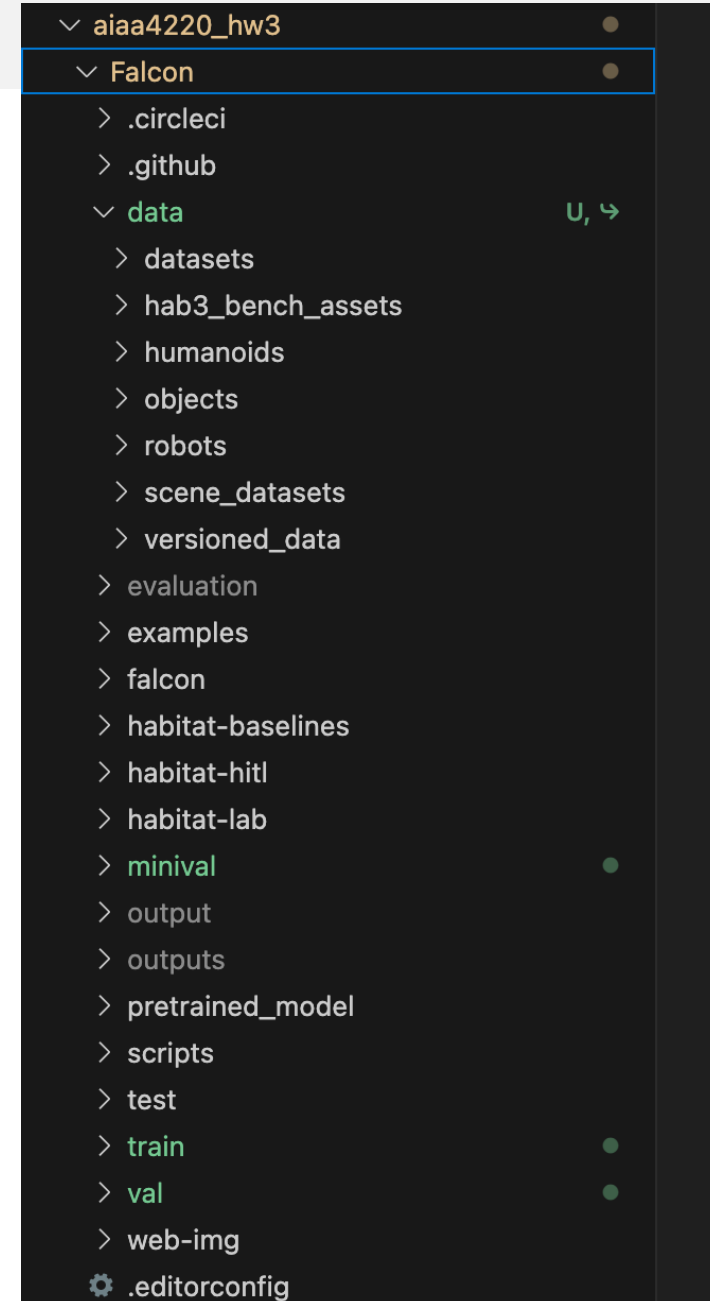
```
mkdir -p data/robots/spot_data  
wget https://github.com/facebookresearch/habitat-  
lab/files/12502177/spot_walking_trajectory.csv -O  
data/robots/spot_data/spot_walking_trajectory.csv
```



Expected File Structure

Put your data under `aiaa4220_hw3/Falcon/`

```
aiaa4220_hw3/Falcon/  
├── data/  
│   ├── datasets/  
│   │   ├── pointnav/  
│   │   │   ├── social-hm3d/  
│   │   │   │   ├── train/  
│   │   │   │   │   ├── content/  
│   │   │   │   │   └── train.json.gz  
│   │   │   │   ├── minival/  
│   │   │   │   │   ├── content/  
│   │   │   │   │   └── val.json.gz  
│   │   │   └── val/  
│   │   │       ├── content/  
│   │   │       └── val.json.gz  
│   ├── scene_datasets/  
│   │   └── hm3d/  
│   ├── robots/  
│   │   ├── hab_spot_arm/  
│   │   └── spot_data/  
│   │       └── spot_walking_trajectory.csv  
│   ├── humanoids/  
│   ├── versioned_data/  
│   └── hab3_bench_assets/  
└──
```



Train

We provide baselines with two model sizes:

- **Full version:** ResNet50 + LSTM512 (requires 24GB+ VRAM)
- **Mini version:** ResNet18 + LSTM128 (requires 6GB+ VRAM)

Take *Mini version* as example, train with following script

```
python -u -m habitat-baselines.habitat_baselines.run \
    --config-name=social_nav_v2/falcon_hm3d_train_mini_junwei.yaml
```

Monitor the training process with *Tensorboard*:

```
# Step 1: Install required dependencies (if not installed)
```

```
pip install six
```

```
pip install tensorboard
```

```
# Step 2: Launch TensorBoard and point to the log directory
```

```
tensorboard --logdir=/app/Falcon/training/falcon/hm3d/tb # or your local path where you clone the code
```

```
# Step 3: Access the TensorBoard dashboard (after running Step 2)
```

```
# Open your browser and visit the URL displayed in the terminal (typically http://localhost:6006)
```

See https://github.com/Zeying-Gong/aiaa4220_hw3#training-commands for more details

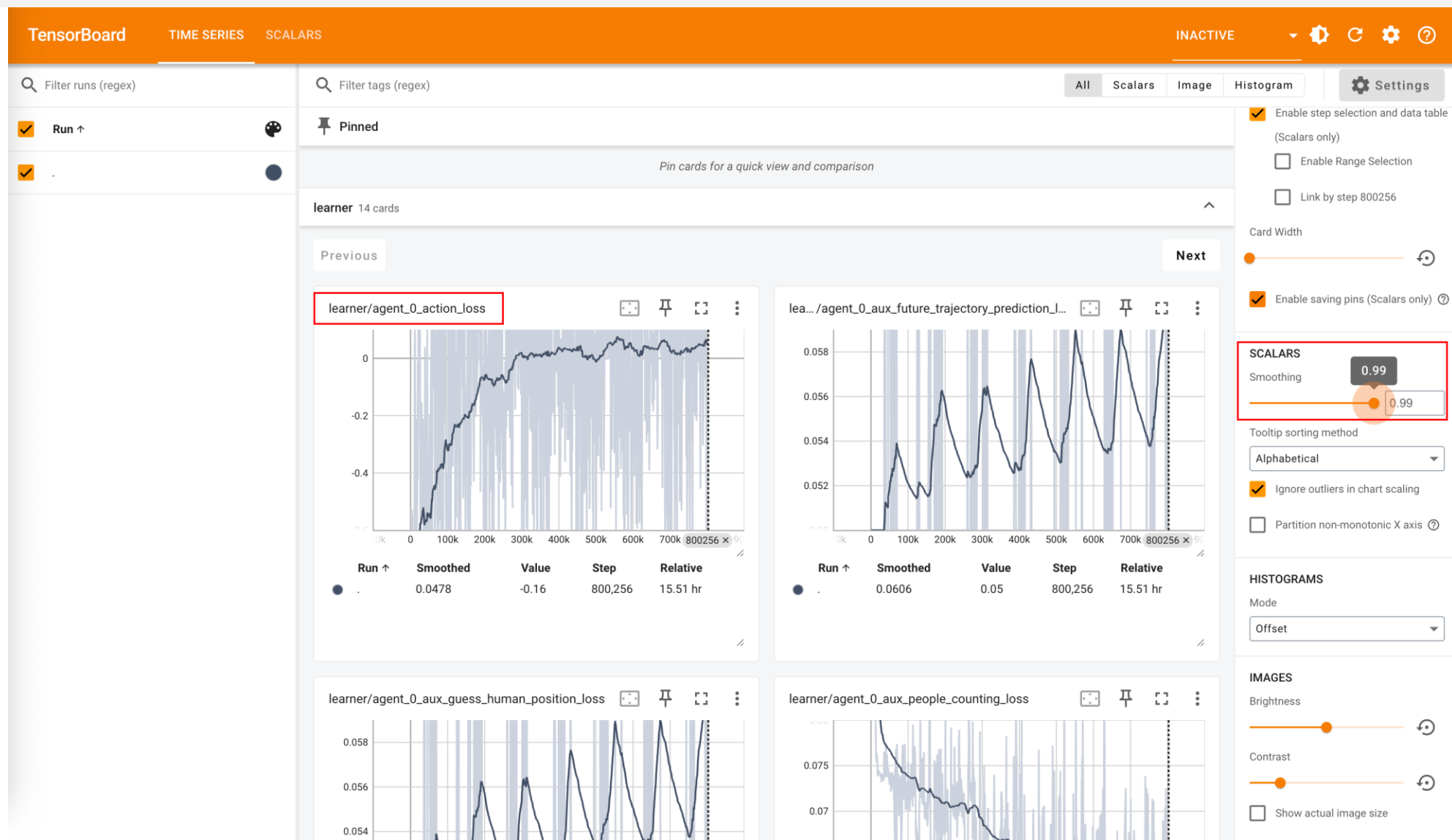
Tensorboard

Run

`tensorboard --logdir .`
in a terminal and `logdir`
refers the directory where
`events.out.tfevents*`
files locate

Then open

<http://localhost:6006/>



Processing Trained Weights

? Why

The training checkpoints contain `aux_loss_modules` which are only used during the training phase. These modules need to be filtered out before the model can be loaded for evaluation or inference.

🔧 How

Using the provided script

```
python process_ckp_for_eval.py \  
evaluation/falcon/hm3d/checkpoints/ckpt.15.pth \  
evaluation/falcon/hm3d/checkpoints/eval.pth
```

See https://github.com/Zeying-Gong/aiaa4220_hw3#processing-trained-weights for more details

Evaluation

Mini Model validation

```
python -u -m habitat-baselines.habitat_baselines.run --config-  
name=social_nav_v2/falcon_hm3d_mini.yaml habitat_baselines.num_environments=4 \  
habitat.dataset.data_path=data/datasets/pointnav/social-hm3d/minival/minival.json.gz \  
habitat_baselines.eval_ckpt_path_dir=evaluation/falcon/hm3d/checkpoints/eval.pth
```

Results you may get

```
2025-10-14 12:27:36,317 Average episode num_steps: 92.6000  
2025-10-14 12:27:36,317 Average episode success: 0.4000  
2025-10-14 12:27:36,317 Average episode human_collision: 0.5000  
2025-10-14 12:27:36,317 Average episode did_multi_agents_collide: 0.5000  
2025-10-14 12:27:36,317 Average episode distance_to_goal: 3.9032  
2025-10-14 12:27:36,317 Average episode distance_to_goal_reward: 0.1182  
2025-10-14 12:27:36,317 Average episode spl: 0.3899  
2025-10-14 12:27:36,317 Average episode psc: 0.9875  
2025-10-14 12:27:36,317 Average episode reward: 2.3053
```

Evaluation Metrics

Success Rate

The percentage of episodes where the robot successfully reaches the goal.

SPL (Success weighted by Path Length)

This metric evaluates path efficiency relative to the shortest possible path.

PSC (Personal Space Compliance)

The percentage of timesteps where the robot remains at least 0.5 meters away from all humans.

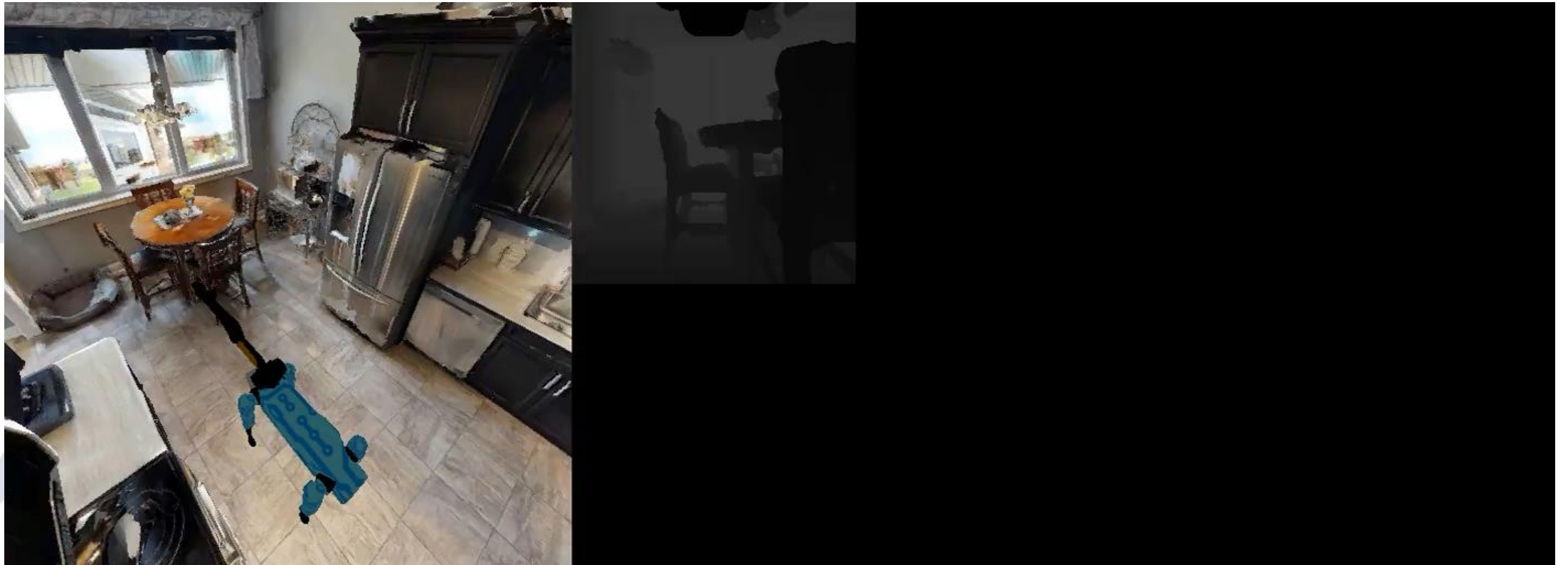
H-Coll (Human Collision Rate)

Percentage of episodes involving any human collision. Any such collision results in failure and affects SR and PSC.

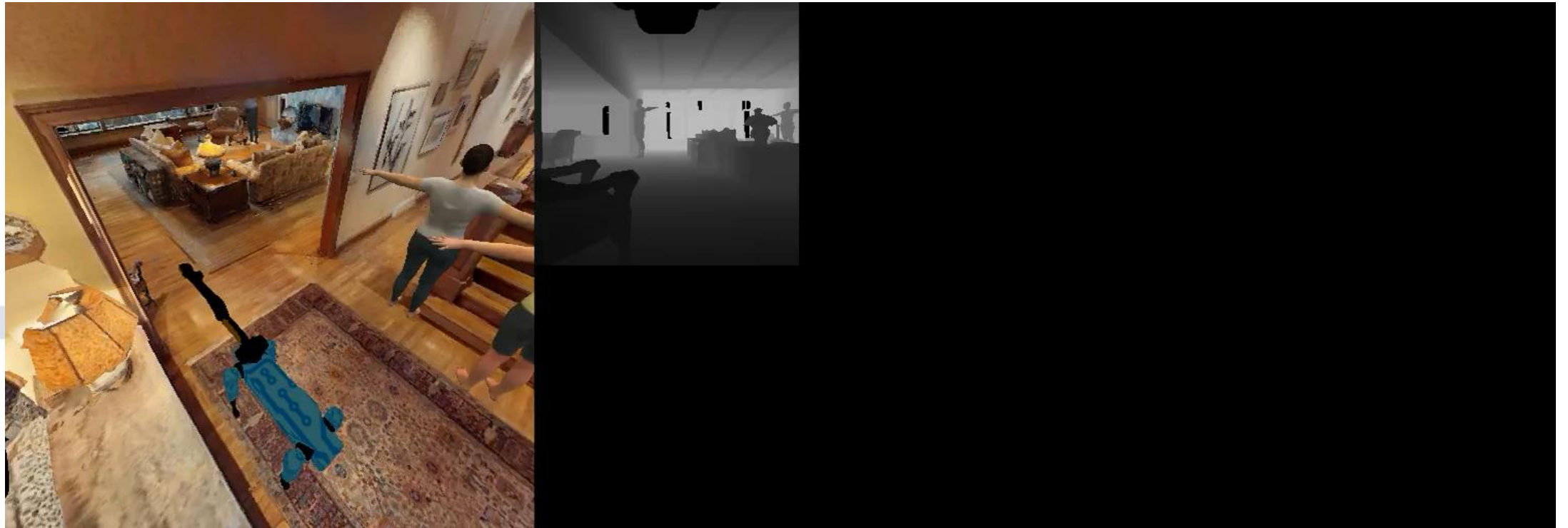
Visualization

Run following script for visualization

```
python process_ckp_for_eval.py evaluation/falcon/hm3d/checkpoints/ckpt.15.pth \  
evaluation/falcon/hm3d/checkpoints/eval.pth
```



Visualization



Submit to EvalAI

EvalAI is an open-source platform for evaluating and comparing machine learning (ML) and artificial intelligence (AI) algorithms at scale.

<https://eval.ai/>



[Dashboard](#)

[Documentation](#)

Evaluating state-of-the-art in AI

EvalAI is an open source platform for evaluating and comparing machine learning (ML) and artificial intelligence (AI) algorithms at scale.

[Host Challenge](#)

[Participate](#)

450+

Hosted AI Challenges

51,000+

Users

500,000+

Submissions

50+

Organizations

EvalAI - Social Navigation Challenge

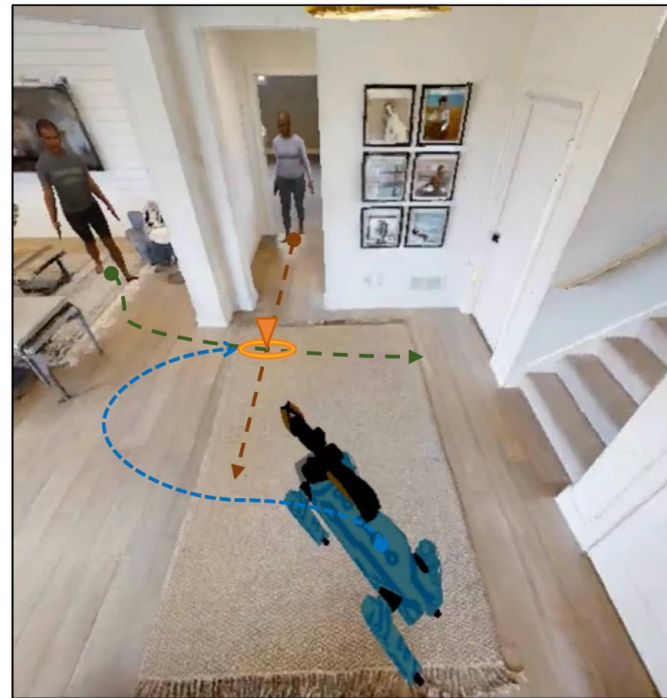
Our project page

Challenge Overview

👋 This is the homework 3 for AIAA 4220 - Introduction to Embodied AI. In this homework we will perform a social navigation task on Habitat platform.

This track challenges students to develop advanced **RGBD-based perception and navigation systems** for autonomous agents operating in **dynamic, human-populated indoor environments**. The goal is to produce agents that navigate **efficiently** while adhering to **social norms**—avoiding collisions, respecting personal space, and adapting to human motion.

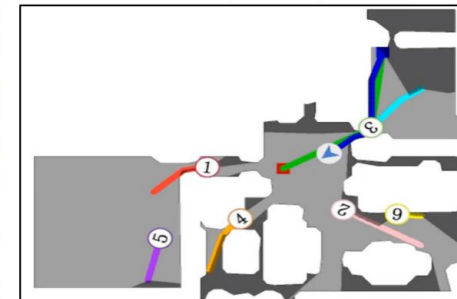
Participants are expected to design algorithms that interpret human behaviors and contextual cues, and generate socially aware navigation strategies. Key challenges include **real-time adaptability**, **occlusion handling**, and **ethical decision-making** in crowded or uncertain scenarios.



(a) Third-person RGB Perspective



(b) Depth Image



(c) Top-down Map

<https://eval.ai/web/challenges/challenge-page/2650/overview>

Submission

1. Download submission example from <https://drive.google.com/file/d/1IRg5iPrWOOKKL6hTCWzZ2deQPLerBAMX/view>

submission_baseline_mini.zip 6 items

Name	Last modified	File size
benchmark	-	-
dataset	-	-
pretrained_model	-	-
task	-	-
falcon_hm3d_minival_mini.yaml	-	-
run.sh	-	-

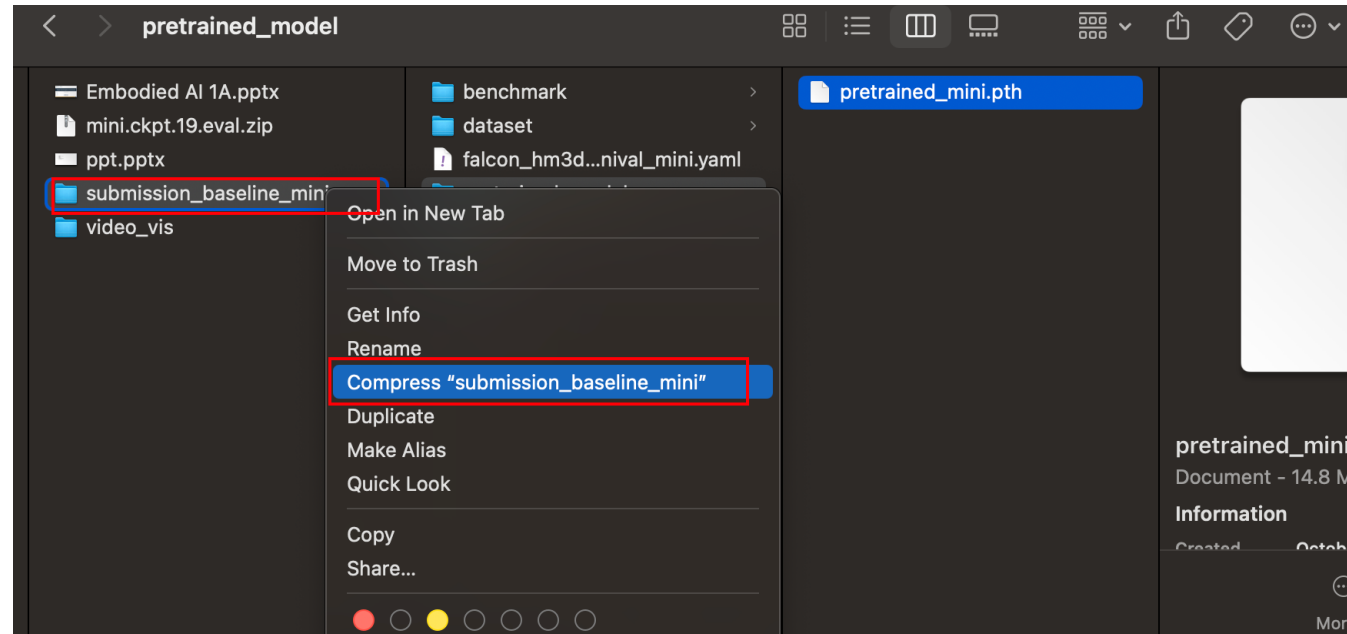
pretrained_model 1 item

Name	Last modified	File size
pretrained_mini.pth	-	-

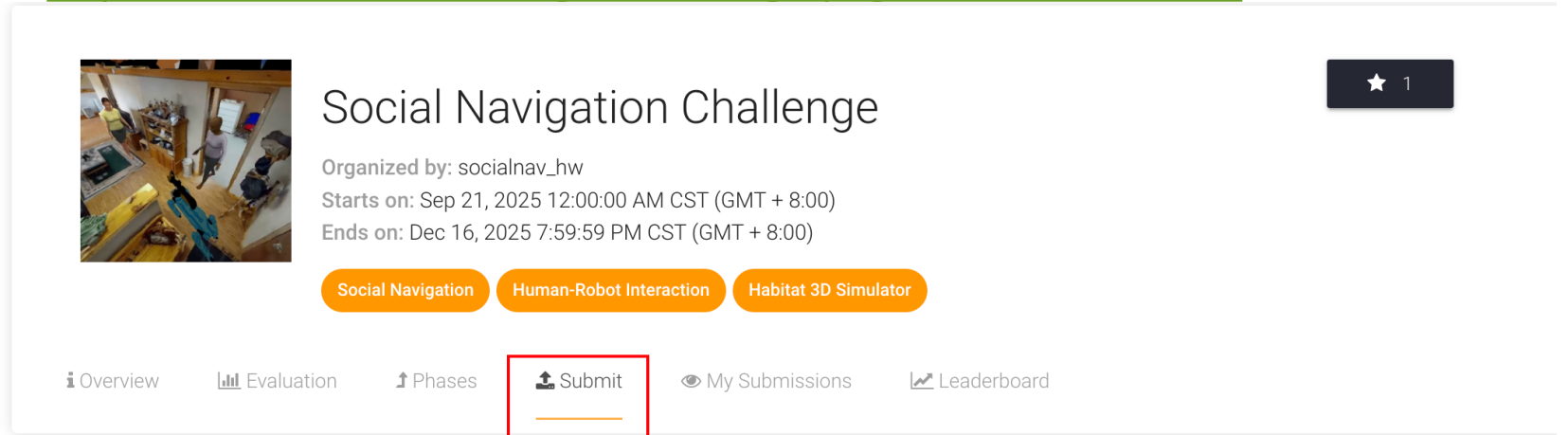
2. replace the `pretrained_model/pretrained_mini.pth` with your processed `.pth` checkpoint file (and it needs to be the **same** file name, otherwise you should modify `falcon_hm3d_minival_mini.yaml`'s `eval_ckpt_path_dir`)

Submission

3. Compress into .zip format



4.1 Submit .zip file to EvalAI, open <https://eval.ai/web/challenges/challenge-page/2650/submission>



Submission

4.2 Roll down and select phase

Make Submission

Select phase:

- ☒ **Phase:** Minival Phase
Start date: Sep 21, 2025 12:00:00 AM
End date: Oct 16, 2025 7:59:59 PM
- ☐ **Phase:** Test Phase
Start date: Sep 21, 2025 12:00:00 AM
End date: Oct 16, 2025 7:59:59 PM

Select submission type:

- ☐ Use CLI (for file size > 400MB)
- ☐ Upload file

Remaining Submissions:

Today (Resets at 12:00 AM UTC): 10

Monthly: 59

Total: 59

Submission

4.3 Submit

- 1. choose submission type
- 2. choose from your local file and upload
- 3. choose public
- 4. select agree
- 5. submit



Select submission type:

- ☐ Use CLI (for file size > 400MB)
- ☒ Upload file

Upload file

mini.ckpt.19.eval.zip

Select submission visibility:

- ☒ Public
- ☐ Private

Method name (Optional)

agree_terms (I confirm this submission was generated fairly and I accept the rules.)*

- ☒ Yes, I agree

Submit

Check your submission

Wait at least 5 min, then
check your submission at
[My Submissions](#)



Social Navigation Challenge

★ 1

Organized by: socialnav_hw
Starts on: Sep 21, 2025 12:00:00 AM CST (GMT + 8:00)
Ends on: Dec 16, 2025 7:59:59 PM CST (GMT + 8:00)

- Social Navigation
- Human-Robot Interaction
- Habitat 3D Simulator

[Overview](#) [Evaluation](#) [Phases](#) [Submit](#) [My Submissions](#) [Leaderboard](#)

My Submissions

My Participated Team:

Minival Phase

File type

Download

Total Submissions: 1

#	Method	Method description	Project URL	Publication URL	Status	Execution time (sec.)	Submitted file	Result file	Stdout
1	None	None	None	None	Finished	450.732597	Link	Link	Link

Rank

You can check your rank at
Leaderboard

We will mainly look
at your results on the
Test split



Social Navigation Challenge



Organized by: socialnav_hw
Starts on: Sep 21, 2025 12:00:00 AM CST (GMT + 8:00)
Ends on: Dec 16, 2025 7:59:59 PM CST (GMT + 8:00)

- Social Navigation
- Human-Robot Interaction
- Habitat 3D Simulator

Overview Evaluation Phases Submit My Submissions **Leaderboard**

Leaderboard

Algorithms are evaluated based on navigation efficiency, social compliance, and collision avoidance. Metrics include SR, SPL, PSC, and H-Coll.

Phase: Minival Phase, Split: Minival Split

Order by metric

B - Baseline * - Private **V** - Verified

Visible Metrics

Rank	Participant team	SR (↑)	SPL (↑)	PSC (↑)	H-Coll (↓)	Total (↑)	Last submission at	Meta Attributes
1	Stargazer (Junwei_checkpoint15_mini2)	0.5000	0.4826	0.8992	0.5000	0.6145	6 days ago	View
2	Host_23006_Team (Baseline_full) B	0.5000	0.4848	0.8936	0.4000	0.6135	20 days ago	View
3	changqing_aiaa4220_test	0.5000	0.4409	0.9347	0.4000	0.6127	2 days ago	View

Presentation

Project 1: Paper Presentation (20%)

You will select 1 paper from our list of Embodied AI research to present. This should show that you understand the paper, and you could present it to students who do not know the paper well.

You have **10 minutes to present and 5 minutes** to handle at least one question (from TA/peers/teacher)

Grade: Instructor (30%), TA (30%), and Peer (40%)



Thanks for your attention!

Changhao Chen
HKUST (GZ)

changhaochen@hkust-gz.edu.cn

Homepage: [changhao-chen@github.io](https://github.com/changhao-chen)